



瞬态噪声以及蒙特卡洛仿真

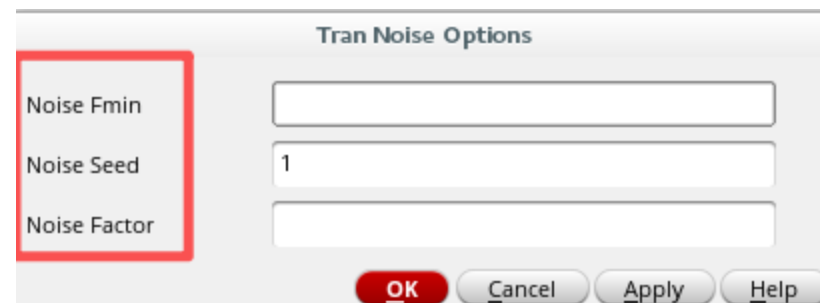
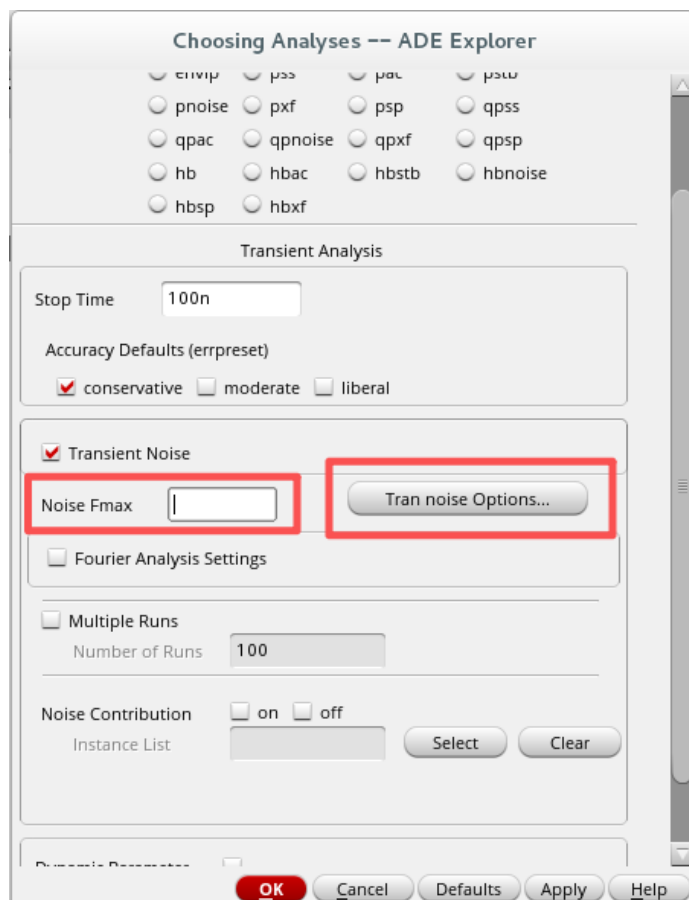
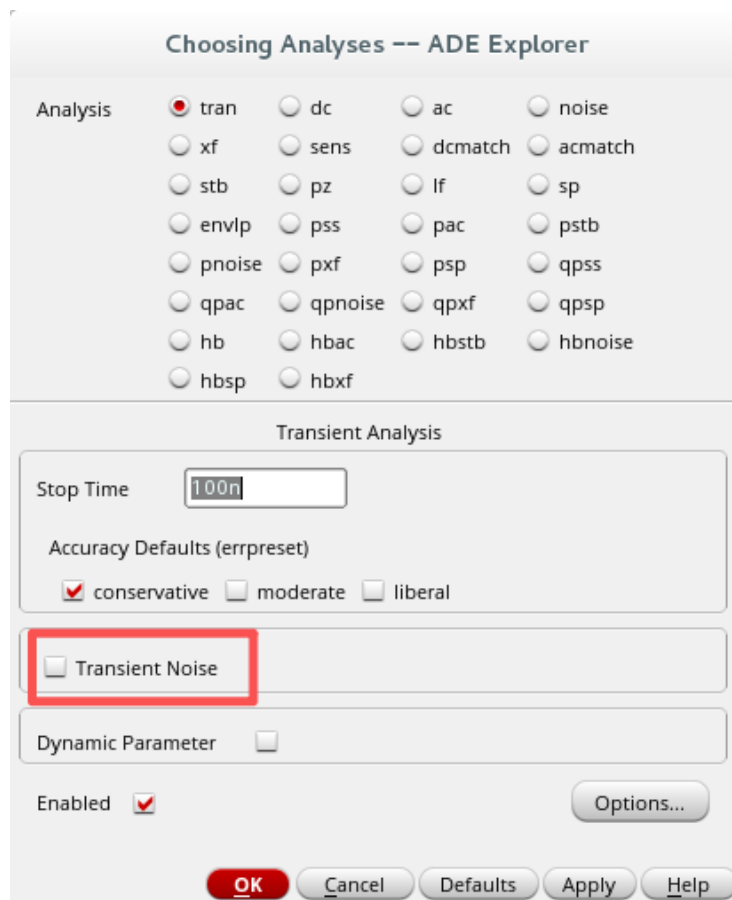
以4bit R-2R DAC为例
集成电路设计工程 -25 秋

- 如何配置tran仿真中的噪声选项
- 以4bit R-2R DAC tran仿真为例进行噪声仿真
- 什么是蒙特卡洛仿真
- 如何进行蒙特卡洛仿真
- 以4bit R-2R DAC tran仿真为例进行蒙特卡洛仿真
- 查看蒙特卡洛仿真结果

如何配置tran仿真中的噪声选项



- 在tran仿真中有这么一个开关：Transient Noise，在平时的仿真中我们都不涉及到它，但是在进行噪声分析的时候我们是需要用到这个开关的，同学们可能发现了在选择tran仿真一栏中也有一个Noise，这个Noise分析主要是在ac下的分析，也就是在频域下的分析，但是如果我们在时域上观察噪声对我们电路的影响的话，就要用到这个Transient Noise了。我们勾选之后发现它有很多的选项，我们暂时先不涉及，只关心这两个选项：Noise Fmax和Tran noise Options,接下来我们将对这4个（ Noise Fmax ， Noise Fmin ， Noise Seed ， Noise Factor）参数进行解释，方便大家理解

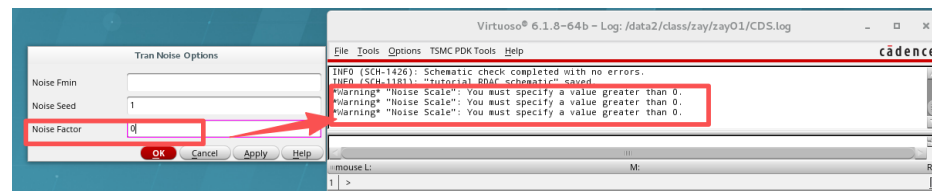


如何配置tran仿真中的噪声选项



noiseamax=0 Hz	The bandwidth of pseudorandom noise sources. A valid (nonzero) noiseamax turns on the noise sources during transient analysis. The maximum time step of the transient analysis is limited to 1/noiseamax.
noiseseed	Seed for the random number generator. Should be positive integer. Specifying the same seed allows you to reproduce a previous experiment.
noiseamin (Hz)	If specified, the power spectral density of the noise sources will depend on frequency in the interval from noiseamin to noiseamax. Below noiseamin the noise power density is constant. The default value is noiseamax, so that only white noise is included by default, and noise sources are evaluated only at noiseamax for all models. 1/noiseamin cannot exceed the requested time duration of transient analysis.
noisescale=1	Noise scale factor applied to all generated noise. Can be used to artificially inflate the small noise to make it visible above transient analysis numerical noise floor, but it should be small enough to maintain the nonlinear operation of the circuit.

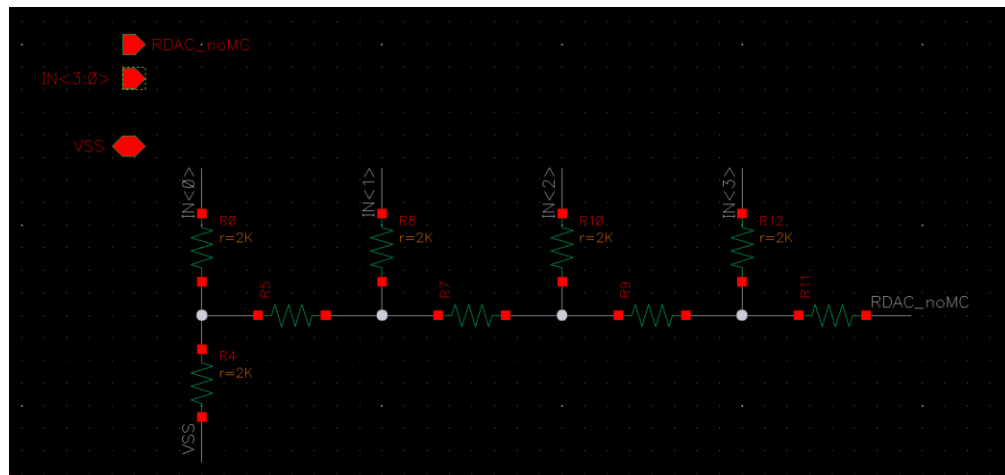
- 这是virtuoso中对这些参数的解释，所以我们在设置选项的时候（只考虑添加白噪声），只需要设置noiseamax和noiseseed这两个即可，noiseamin使用default值即可。（可以思考一下为什么），这里建议noiseamax设置 $\geq 10G$ ，noiseseed取正整数即可。
- 我们发现在noise option中有一个noise factor的设置，但是并没有noisescale的设置，但是如果你在noise factor中输入0，在CIW界面会显示一个warning，所以这两个参数应该是版本更新之后更改了参数名称。



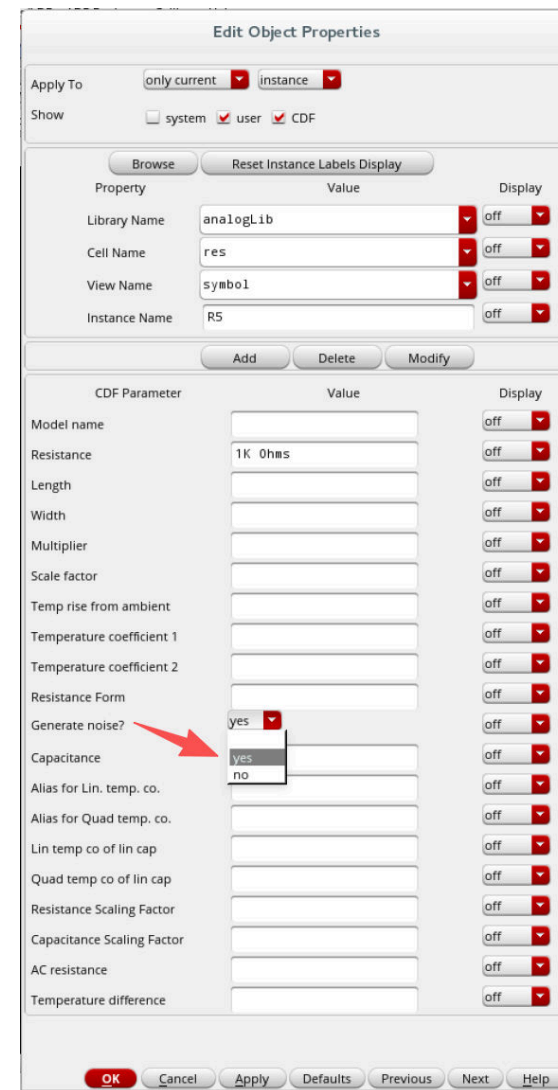
以4bit R-2R DAC为例展示打开噪声开关的仿真效果



- 以analoglib中的res搭建4 bit R-2R电路，其中 $R = 1K\Omega$ ，（打开res的设置界面，可以看到有一个Generate noise?的选项，勾选yes，否则理想的电阻是没有噪声的，即使是打开了噪声仿真开关）



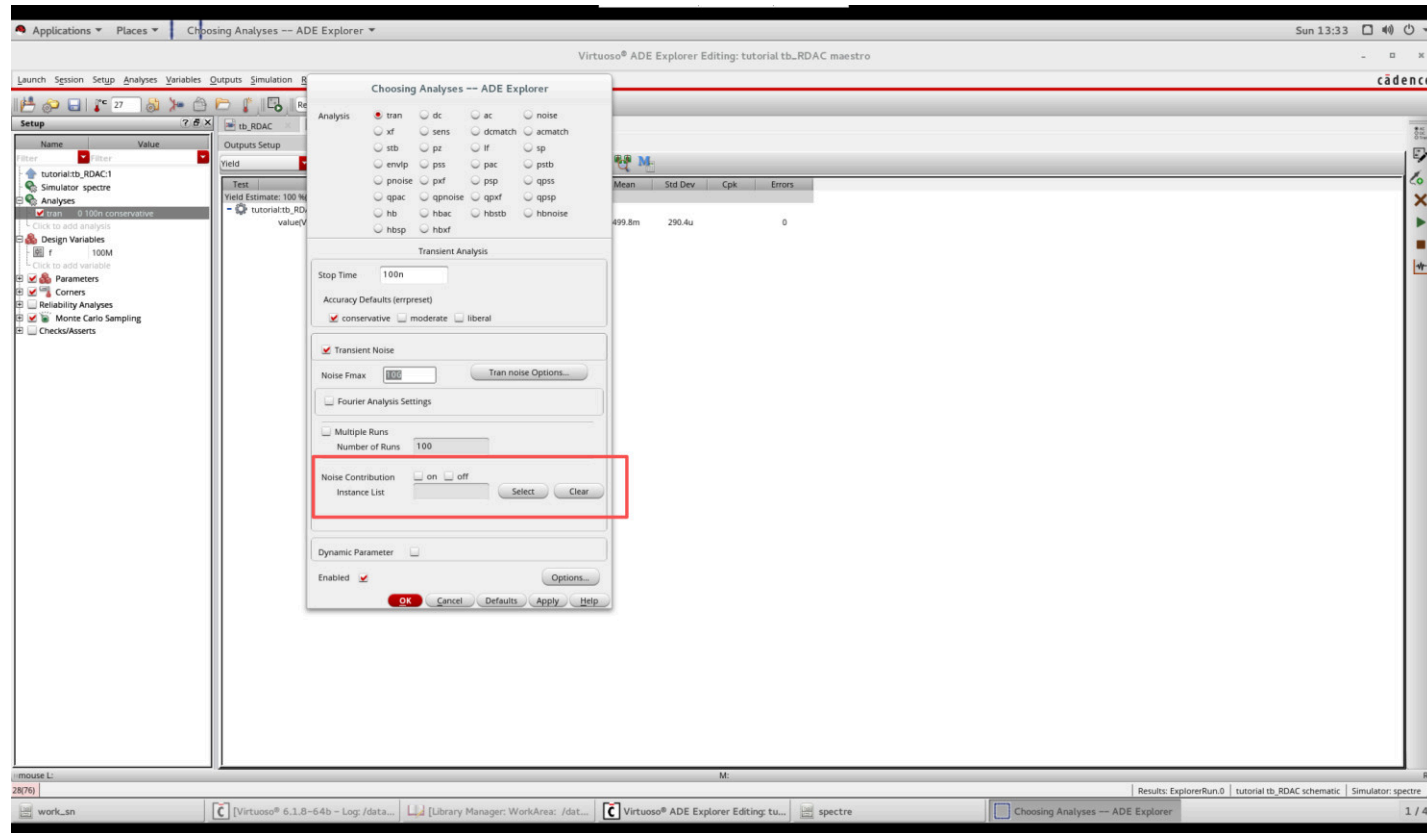
大家要养成电路和tb区分开，养成一个好的习惯，方便电路阅读也方便后续debug，即使是很小的电路也有必要把它制作成symbol



以4bit R-2R DAC为例展示打开噪声开关的仿真效果



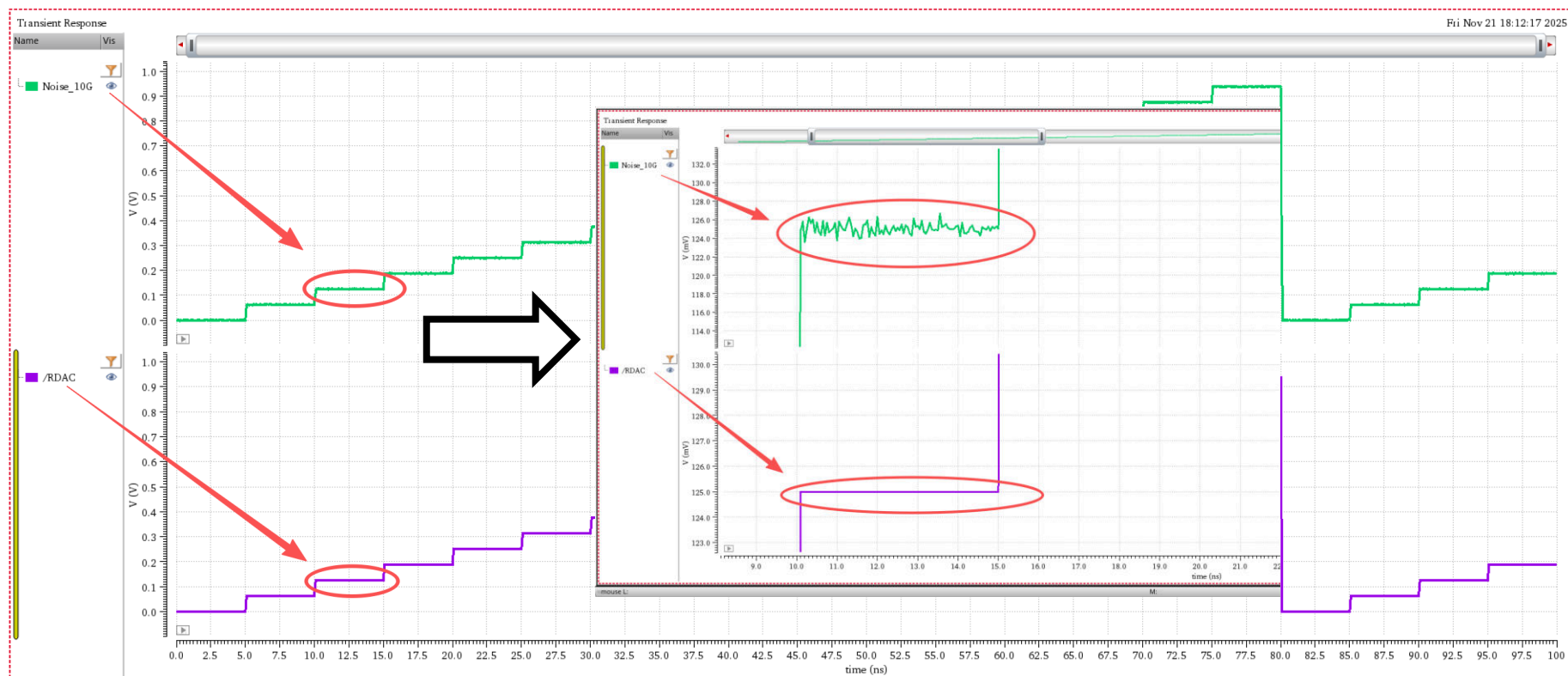
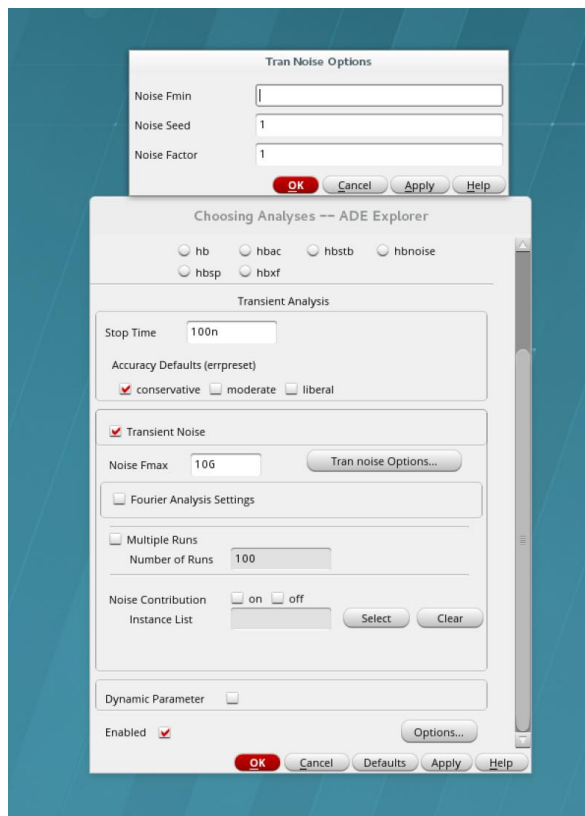
- 实际的R-2R的输入端是有buffer，也就是说在IN<3:0>是输入在buffer的输入端，而buffer的输出端才接到电阻端，（思考一下为什么会这样？），但是本教程只为了进行噪声仿真的说明，所以忽略了该buffer，感兴趣的同学们可以仿真一下带buffer的噪声情况，实验一下buffer的尺寸和输出端噪声的关系，并可以对比有Buffer参与的R-2R DAC的输出噪声和没有buffer的输出的区别，这里有一个小的选项可以早进行噪声仿真的时候，选择特定模块产生或者不产生噪声。



4bit R-2R DAC有无噪声的仿真结果



- 在该设置下，我们可以看到有噪声和无噪声时候的RDAC输出的值的不同，体现了噪声的影响，所以在实际设计的时候，如果你的电路是对噪声敏感的电路，一定要做噪声仿真进行分析，因为真实的物理世界是充满噪声的！大家学会了Transient Noise的仿真，请完成一下作业2中的随机数仿真，体会一下有噪声的仿真。



什么是蒙特卡洛仿真？



蒙特卡洛的定义：一种基于**统计学**和**随机采样**的数值计算方法。

(取名自著名的蒙特卡洛赌场，利用随机性来解决复杂问题。)

在电路设计中的核心逻辑：1.**放弃“单点预测”**：不再假设所有器件都是完美的标称值。2.**采用“概率分布”**：根据代工厂提供的统计模型，随机改变电路中每个器件的参数。3.**大规模模拟**：通过运行成千上万次仿真，预测电路性能的分布情况。4.**原理图中的器件是完美的**。但是**实际的制造**充满了不确定性，不存在两个完全一样的晶体管，电阻，电容等。

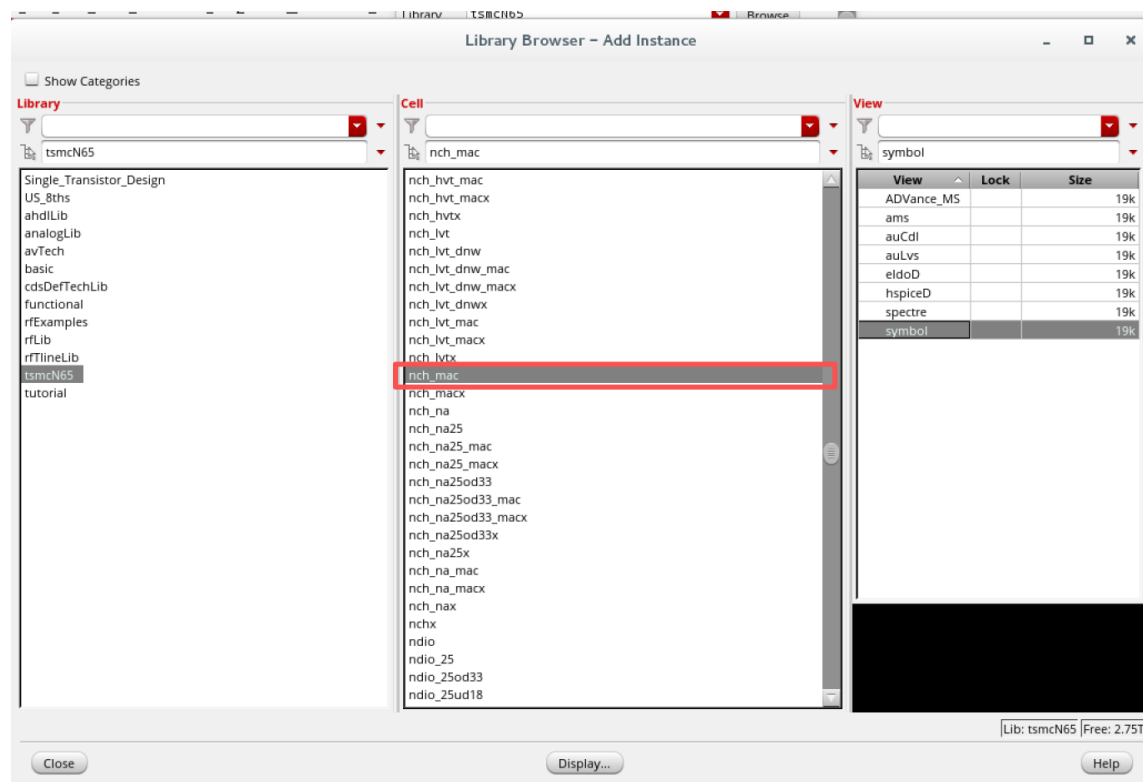
3-Sigma (3σ) 设计原则

- 目标：99.73% 的采样点落在设计规格范围内。
- 蒙特卡洛通过大量采样（如 1000 次+），直接模拟出良率数据。

工艺角仿真和蒙特卡洛仿真的区别

	Corner Analysis (工艺角)	Monte Carlo (蒙特卡洛)
方法论	确定性 (Deterministic)	统计性 (Statistical)
测试范围	仅测试极端的性能边界 (SS, FF)	测试参数的全高斯分布范围
主要功能	确保电路在最坏条件下能工作	计算良率 (Yield)，量化失配
准确度	往往过于悲观，忽略相关性	更接近真实的量产情况

首先需要选择可以进行蒙特卡洛仿真的器件（最重要的步骤之一）什么是可以进行蒙特卡洛仿真的器件，例如：PDK中的nch_mac，为什么我们说它是可以进行蒙特卡洛仿真的，我们来看他的spice网表，可以发现，nch_mac是在nch模型基础上加了许多参数，所以在进行蒙特卡洛仿真的时候这些参数的值会变化使得其仿真结果发生变化。

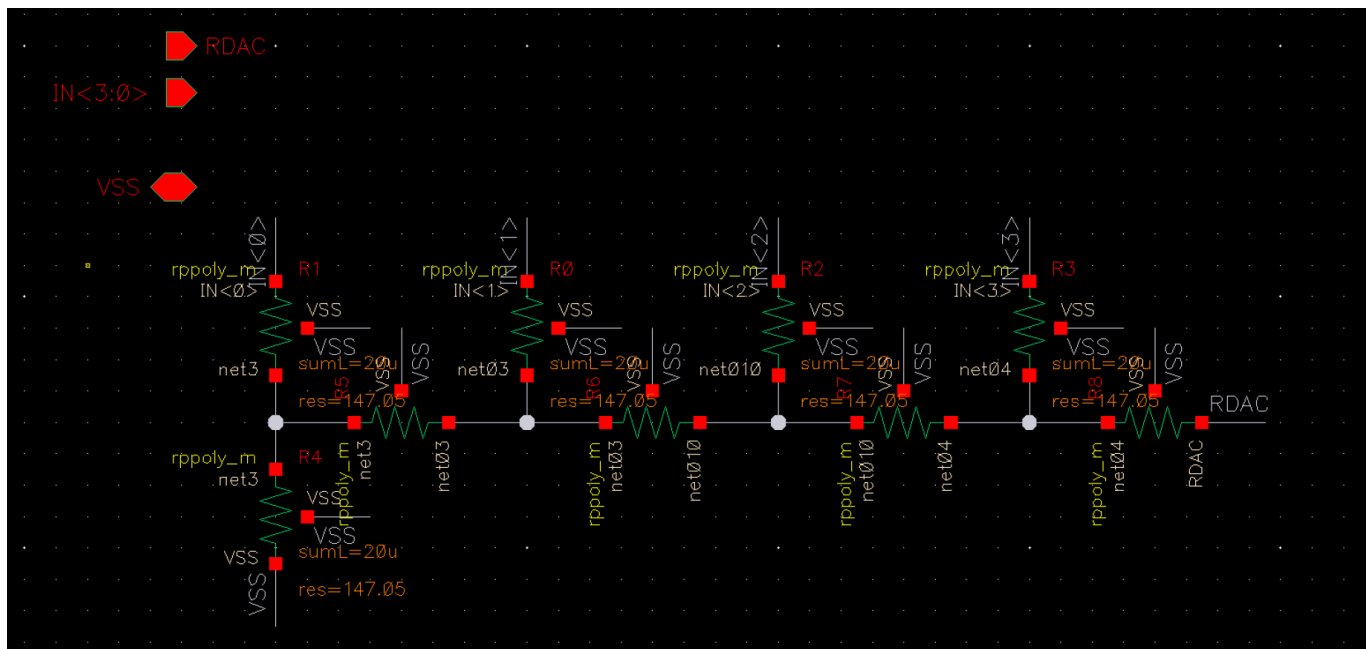


```
section mos
// *****
//
//      *
//                               1.0V STD-VT NMOS DEVICE MODELS
//
//      *
//
//      *
// *****
// *****
// Mismatch_model *
// *****
inline subckt nch_mac ( nl n2 n3 n4 )
parameters l=lu w=w multi=1 nf=n1 factor=1 sigma=1 dmcg=7.15e-008 xw= + 1.6e-08
+ dxwn ad=2 * dmcg / factor * (w / nf + xw / factor) as=2 * dmcg / factor * (w
+ / nf + xw / factor) pd=4 * dmcg / factor + 2 * (w / nf + xw / factor) ps=4 * dmcg
+ / factor + 2 * (w / nf + xw / factor) nrdd=dmcg / factor / (w / nf + xw / factor)
+ nrs=dmcg / factor / (w / nf + xw / factor) sa=0.405e-6 / factor sb=0.405e-6 /
+ factor sd=2 * dmcg / factor sca=sca=9e-6 / factor scb=scb=9e-6 / factor scc=scc=9e-6 / factor
+ mismatchflag=1 temp1=parrn1 temp2=parrn2 fac1=0.443742 * templ * sigma + 0.896155
+ * temp2 * sigma fac2=0.443742 * templ * sigma - 0.896155 * temp2 * sigma lef=l
+ * factor - 1.46e-08 wef=w / nf * factor + 1.6e-08 geo_fac=1 / sqrt(multi * nf
+ * lef * wef * le12) vthmism=0.00235886 * geo_fac * fac1 * mismatchflag dlmsin=0.00171632
+ * geo_fac * lef * fac2 * mismatchflag dwmsin= - 0.00171632 * geo_fac * wef * fac2
+ * mismatchflag toxmisin=0.00171632 * geo_fac * toxin * fac2 * mismatchflag tnoiax=-1.603e05
+ / (l * le6) + 2.013e07 rnoiax=0.011 / (l * le6) + 0.4433
nch_mac ( nl n2 n3 n4 ) nch w=w l=l ad=ad as=as pd=pd ps=ps nrdd=nrd nrs=nrs m=multi
+ nf=nf sa=sa sb=sb sd=sd sca=sca scb=scb scc=scc
// *** mismatch ***
model nch bsim4 {
    l: type=n
        + lmin=9.999997e-006-(dxln+dLmsin)          lmax=2.000001e-05
        + wmin=1e-009-(dxwn+dwmisn)                 wxmax=0.0009000001   version=4.5
        + gbmin=1.00e-12                             rbsb=50              ud=0
        + rbdb=50                                     rgeomod=1                rbpb=50
        + rshg=15.953                                rbps=50                  rbpd=50
        + ngcon=1                                     binunit=2               paramchk=1
        + mobmod=0                                    capmod=2                igcmod=1
        + igbmod=1                                    diomod=1                rdsmod=1
        + rbodmod=0                                   rgatemod=1*rgflag       permod=1
        + acqsgmod=0                                  trngsmode=0             tempmod=0
        + ton=25                                       tox=(toxn+toxmisn)      toxin=2e-009
        + dtov=2.4005e-010                          opcv=0                  viat=0
}
```

如何进行蒙特卡洛仿真



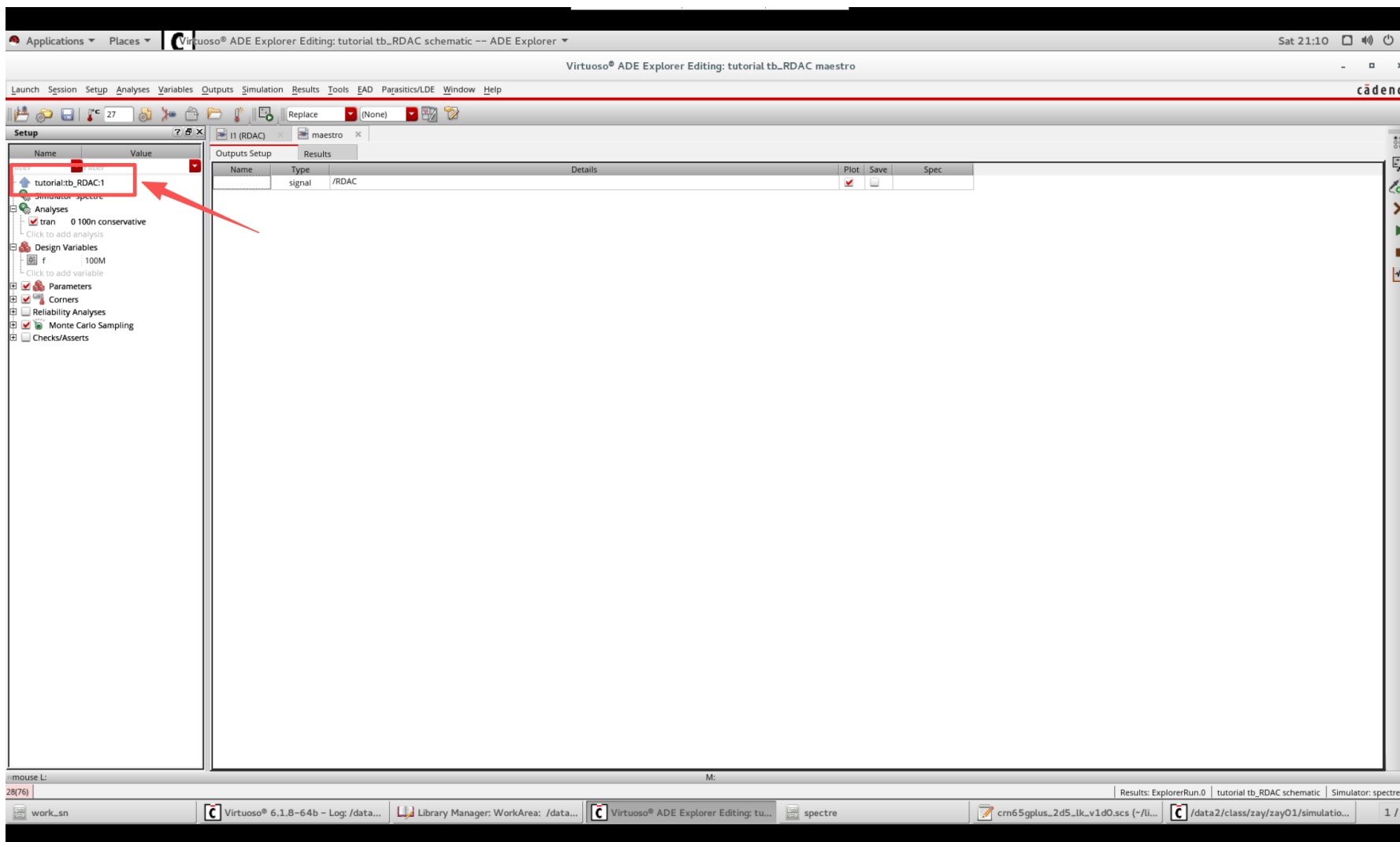
由于我们使用的是理想的电阻，所以在仿真的PDK中是肯定没有有关mismatch的描述的，所以我们需要更换一个PDK中的电阻，我们把电阻更换为rppoly_m，仍然保持R-2R的结构（这里不纠结阻值大小，保证比例满足功能即可）
这样我们完成了蒙特卡洛仿真的第一步



如何进行蒙特卡洛仿真



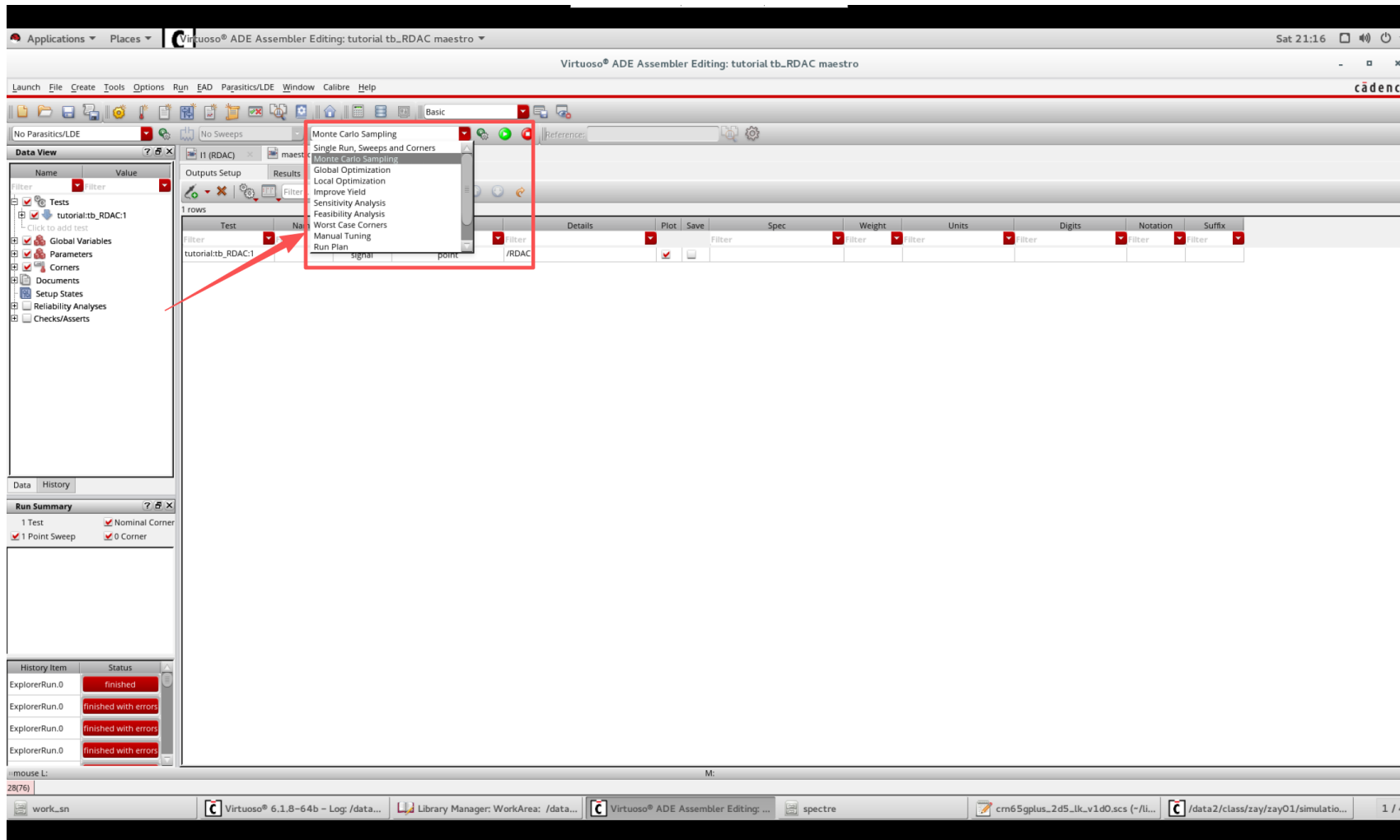
接着我们打开之前设置的ADE Explore，回到仿真设置的顶层（点击这个向上的箭头）



如何进行蒙特卡洛仿真



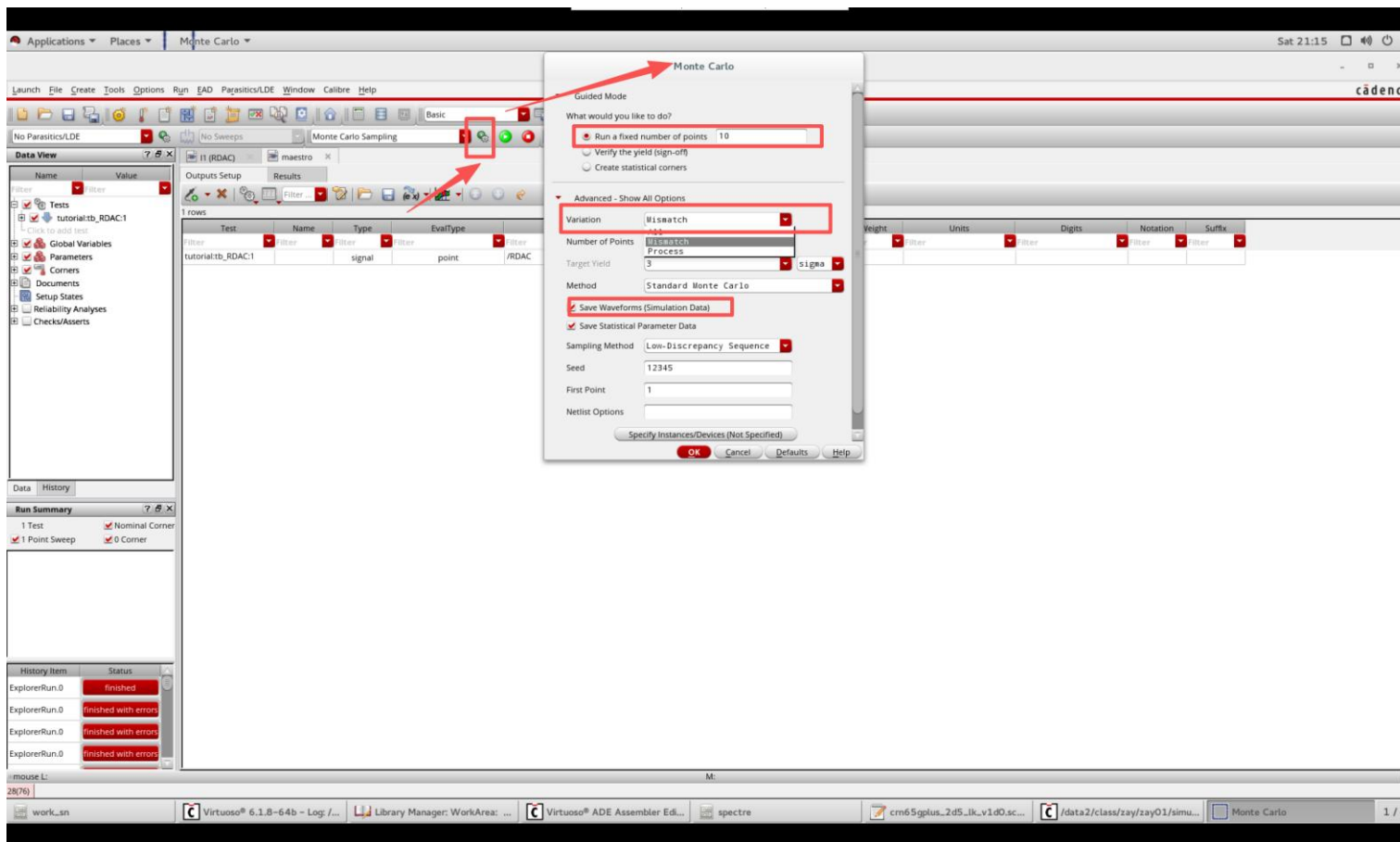
回到仿真界面的顶层之后，在红色箭头所指向的地方，打开仿真模式的菜单，选择第二个Monte Carlo Sampling，选择好这个之后意味着之后的仿真就是要进行蒙特卡洛仿真了



如何进行蒙特卡洛仿真



然后点击红色框选的带有小齿轮的图标，就打开了Monte Carlo的选项菜单，我们重点关注红色框选的三个选项，第一个是选择你要进行采样的点数，一般采样点数越大结果越精确，但是耗费的时间和资源就会越多，一般建议200~500个采样点即可，之后第二个红框选择的是你所要运行的仿真中考虑的因素，是mismatch还是process或者都有，这里一般建议选mismatch，当然还是根据电路需求进行选择，最后的红色方框是保存仿真波形数据要勾选。

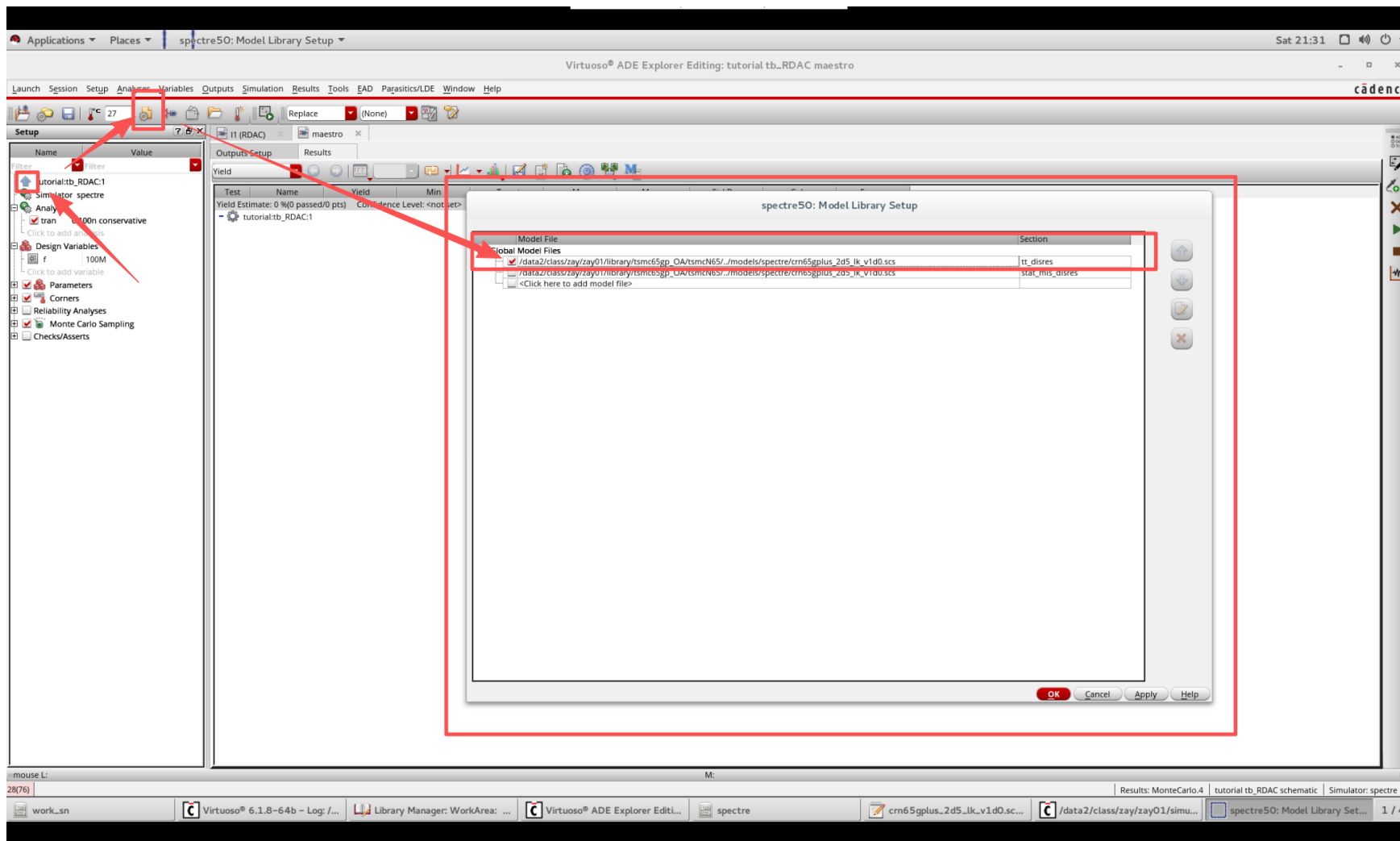


Mismatch和process的区别是，process表示所有元器件的整体上的工艺参数的随机波动，而mismatch指的是不同元器件之间的随机波动而造成的失配。这里因为只做演示，所以选取100个采样点，x个采样点的意思就是会运行x次仿真，并且每次仿真的元器件的参数会不同，但是会遵循正态分布。

如何进行蒙特卡洛仿真



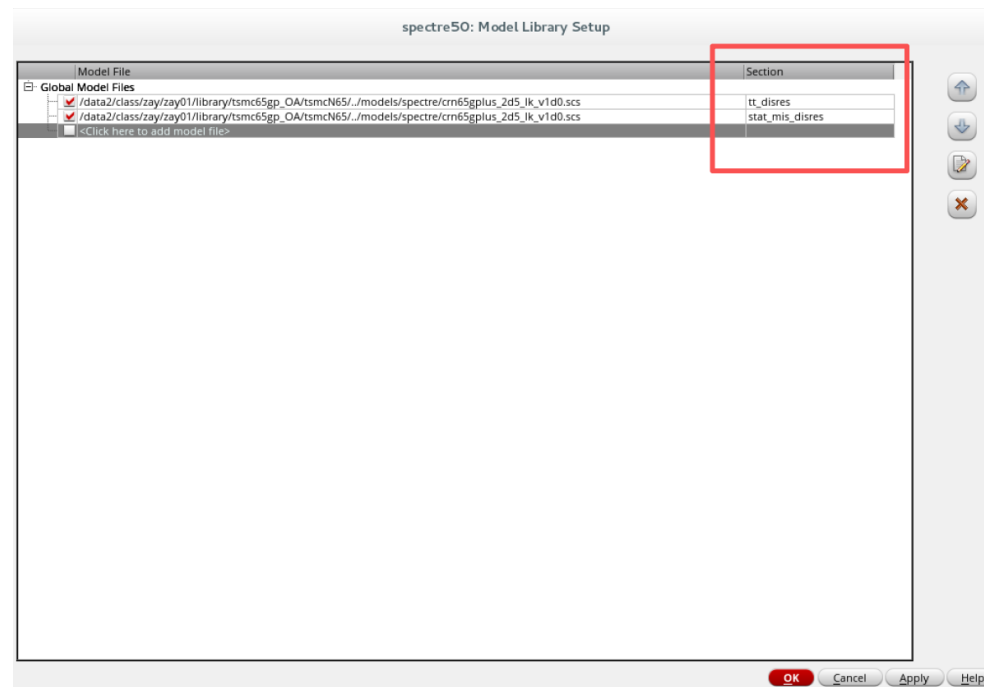
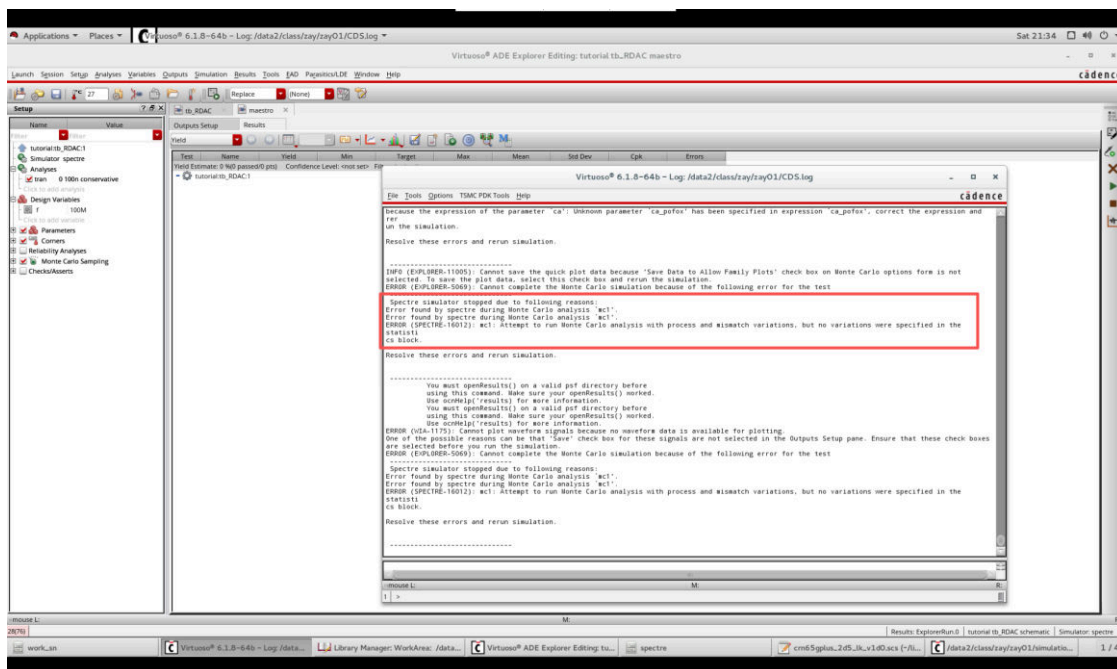
再跑仿真之前的最后一步也是最重要的一步，加蒙特卡洛仿真所需要的文件（这里点击箭头回到库文件添加的界面），这里仿真res，首先需要添加的就是再spice文件中描述res部分—tt_disres，但是此时添加了该仿真文件后，如果点击run按钮，在CIW中会出现这个ERROR



如何进行蒙特卡洛仿真



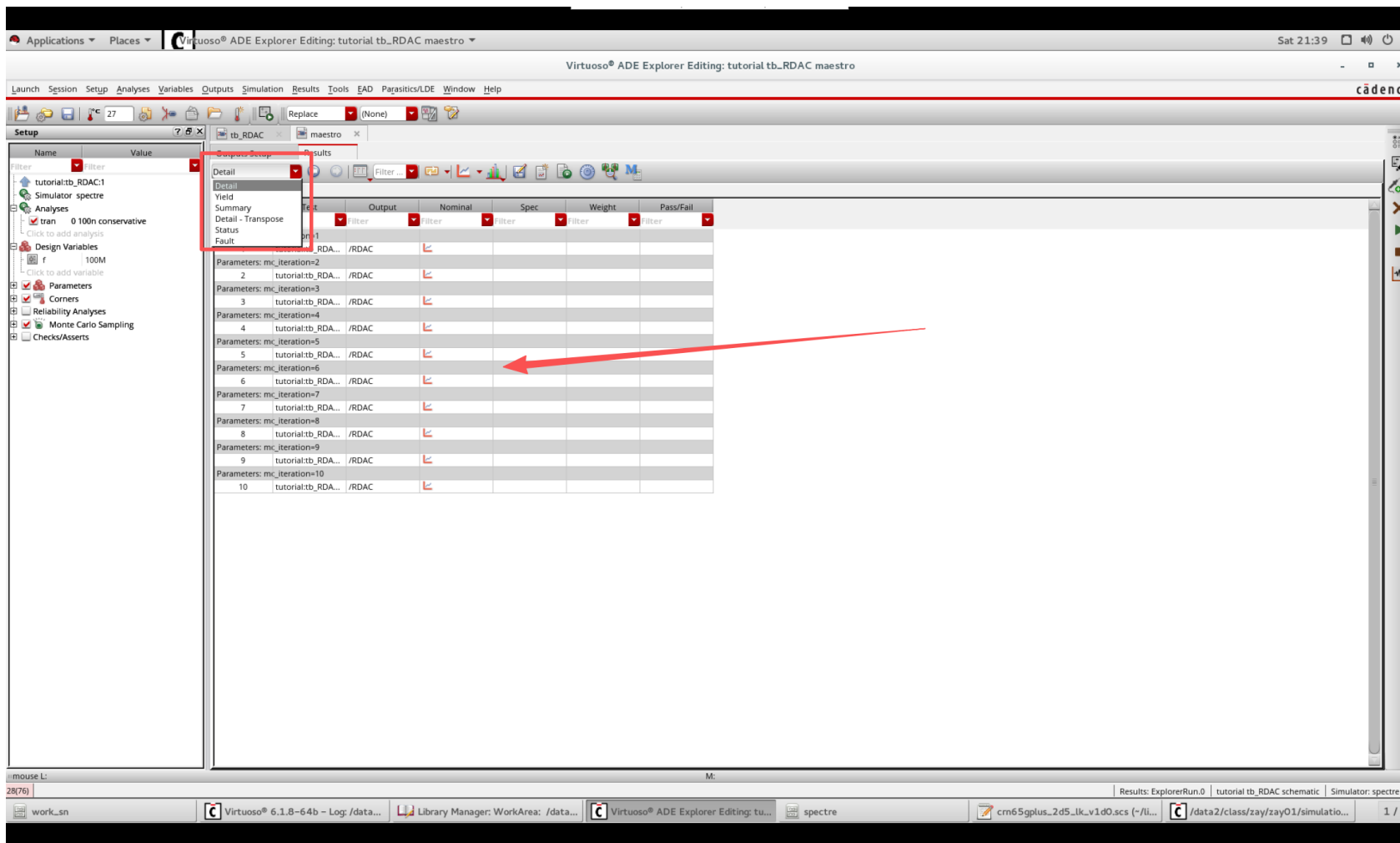
再跑仿真之前的最后一步也是最重要的一步，加蒙特卡洛仿真所需要的文件（这里点击箭头回到库文件添加的界面），这里仿真res，首先需要添加的就是再spice文件中描述res部分—tt_disres，但是此时添加了该仿真文件后，如果点击run按钮，在CIW中会出现这个ERROR：找不到mc的仿真文件，所以要添加上有关mismatch的仿真文件，stat_mis_disres，如果在你的电路中有添加了例如mos管或者二极管在整体电路中，那么请记住添加上他们的有关描述mismatch的仿真文件，命名一般相同



以4bit R-2R DAC tran仿真为例进行蒙特卡洛仿真



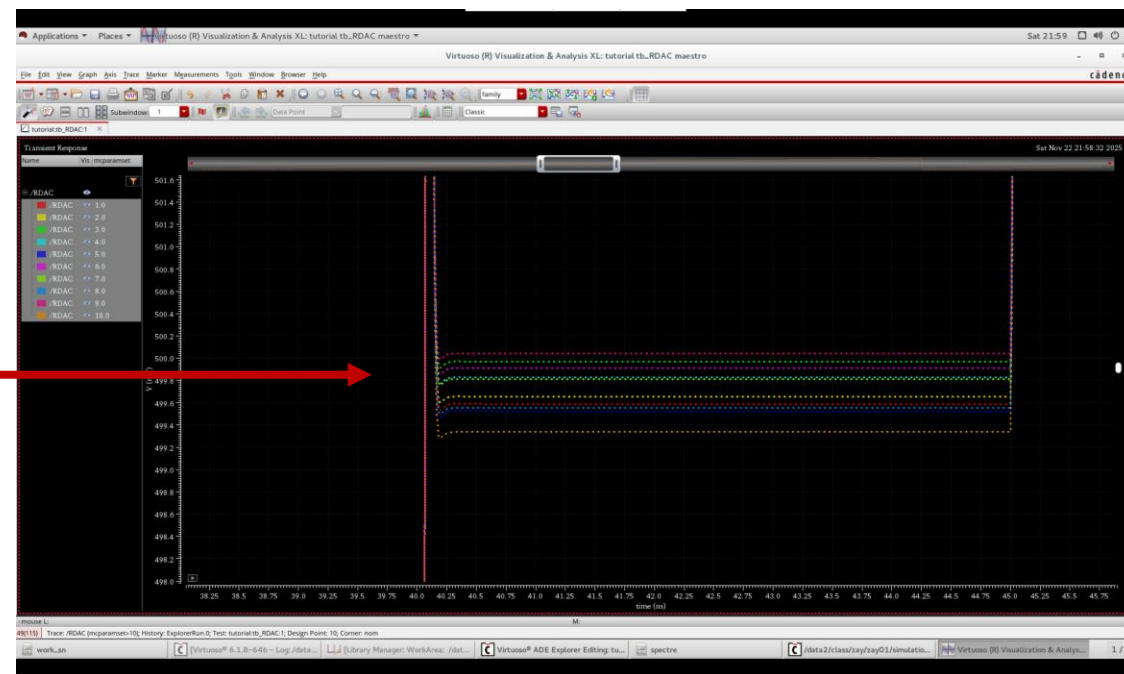
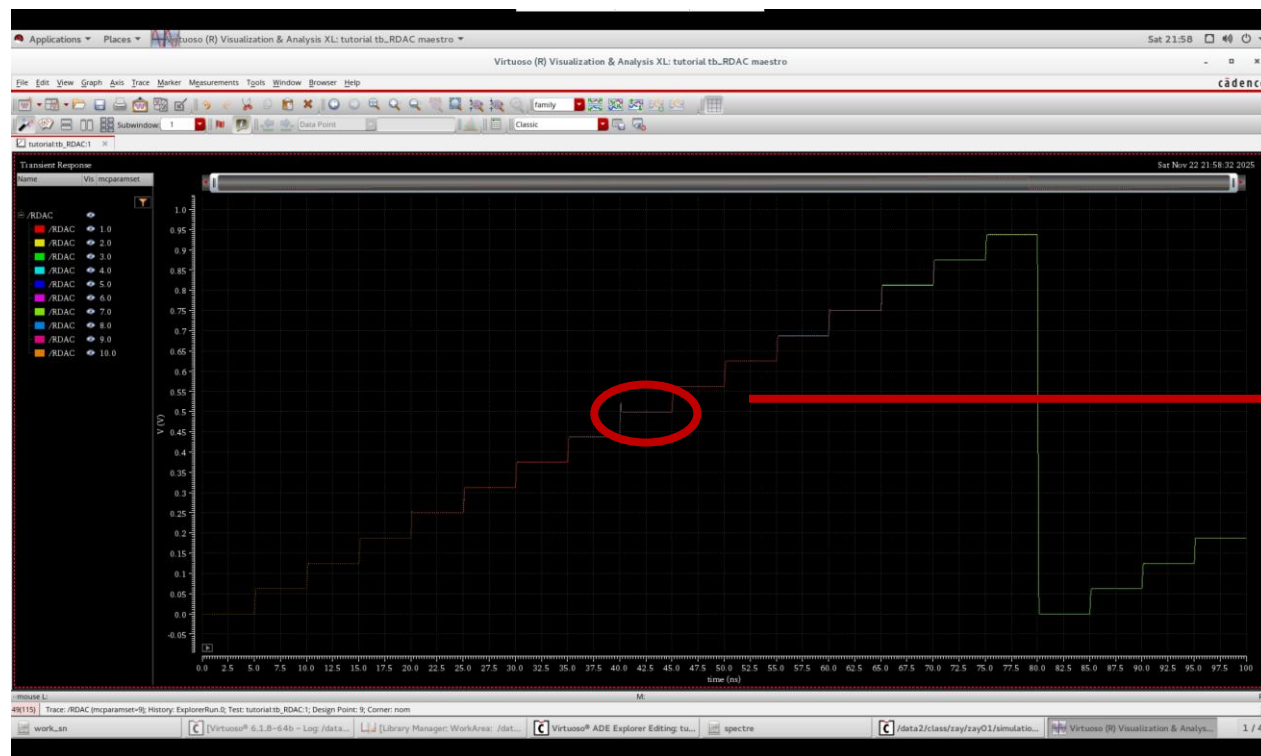
选择好文件之后，就可以点击run按钮进行仿真，我们可以看到一共会有10个仿真在进行，（注意：在点击run按钮之后，界面会自动调到Yield界面，点击这个菜单，选择Detail就会看到仿真界面），现在看到仿真已经完成，我们把它们都polr出来看一下



查看蒙特卡洛仿真结果



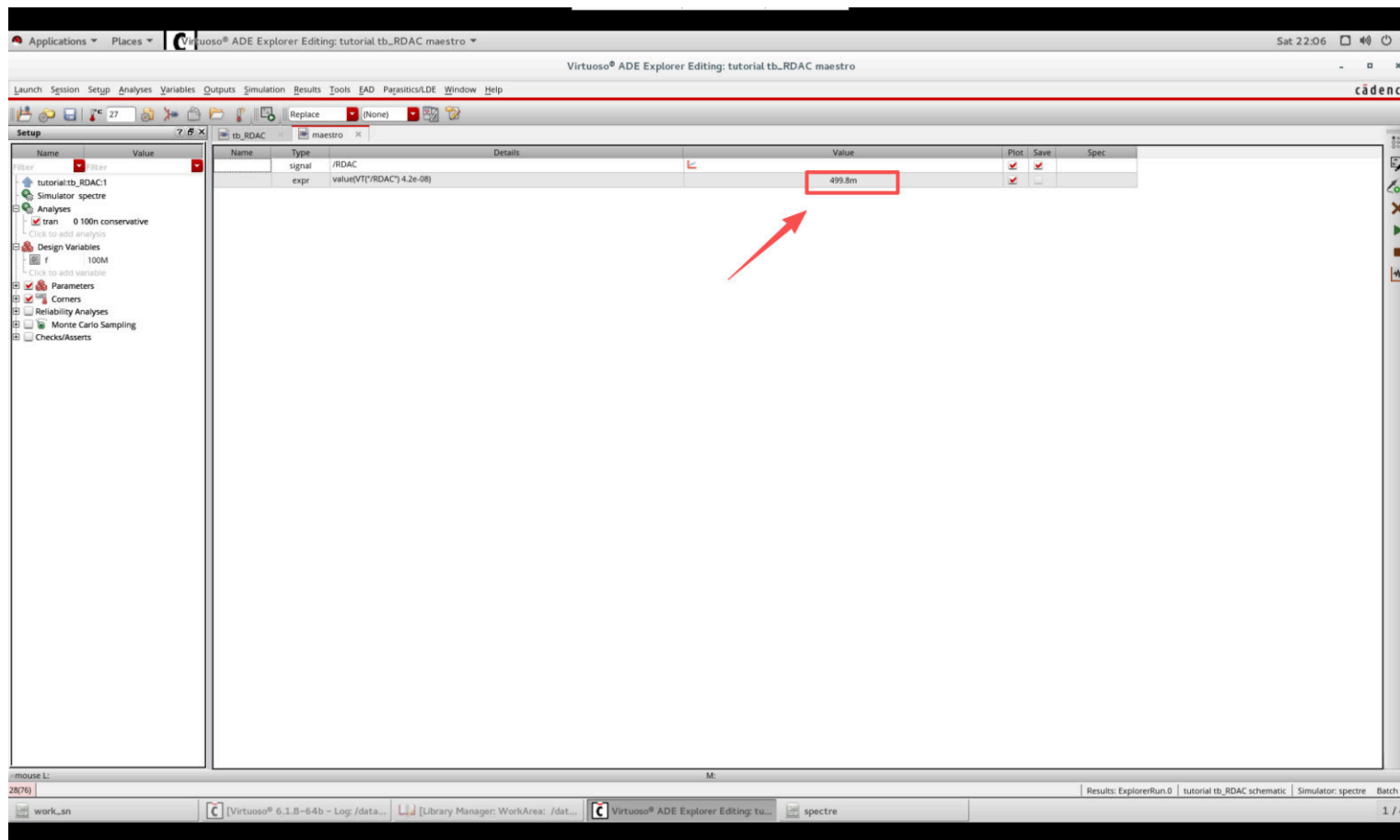
选择好文件之后，就可以点击run按钮进行仿真，我们可以看到一共会有10个仿真在进行，（注意：在点击run按钮之后，界面会自动调到Yield界面，点击这个菜单，选择Detail就会看到仿真界面），现在看到仿真已经完成，我们把它们都plot出来看一下，我们可以发现，变成了一条曲线簇，那么这个曲线簇代表着你的电路有可能出现的情况，那么就要注意在这个范围内，你的电路是否还满足要求。



查看蒙特卡洛仿真结果



刚刚是使用蒙卡查看整个仿真过程波形的变化，但是这样虽然宏观但不够具体，比如有时候我们需要了解的是我们电路中的指标它具体在什么范围内，所以我们还需要查看电路中某个具体的指标的统计直方图，这通常是蒙卡所展示的结果，我们保存这4bit R-2R DAC在输入1000时候的具体电压值，这里简单的选择42ns时候的RDAC输出值，我单独运行一次tran仿真，可以看到该数值为499.8m，非常接近理想的500mV，那么它在mismatch的影响下的值会是什么样呢，让我们进行蒙卡仿真吧！



查看蒙特卡洛仿真结果



我们重复之前的操作，但是这次我们把采样点设置为200，，（思考为什么采样10个点不行？如果要看到 $\pm 3\sigma$ 需要多少个点？）。之后我们保持在Yield界面，我们可以看到如下结果，如果你勾选了plot会自动显示直方图结果，如果没有的话，点击下面的这个结果，然后右键选择Histogram，你就可以得到这个值的均值和方差了，然后根据统计特性取估算你得电路是否满足指标！到这里，所有的蒙特卡洛仿真就结束了，希望大家在后续的学习中多体验蒙特卡洛仿真对电路设计的裕度考量的指

